U21ADP05 – EXPLORATORY DATA ANALYSIS AND VISUALIZATION
ASSIGNMENT - II

PROJECT REPORT

COVID-19 PATIENT RISK ANALYSIS

Submitted by

ABISHEK P

25ADL01

Artificial Intelligence and Data Science

III – AIDS B



KPR Institute of Engineering and Technology

Learn Beyond (Autonomous, NAAC "A")

Avinashi Road, Arasur, Coimbatore.

ABSTRACT

COVID-19 Patient Risk Analysis is a data science and machine learning project designed to examine patient characteristics and estimate the risk of an adverse COVID-19 outcome. The workflow includes dataset inspection, exploratory data analysis, data cleaning, missing-value and duplicate handling, outlier review, categorical encoding, and numerical scaling where appropriate. A classification model is trained using separate training, validation, and test partitions. Performance should be assessed with precision, recall, F1-score, ROC-AUC, and a confusion matrix, with particular attention to missed high-risk cases. The supplied Colab notebook could not be inspected because it requires Google sign-in; therefore, this report is a project-ready draft, and its dataset details, model settings, charts, and measured results must be verified against the executed notebook. This analysis is educational and is not a clinical decision tool.

1. INTRODUCTION AND OBJECTIVES

COVID-19 outcomes can vary with age, underlying health conditions, symptoms, and other patient factors. Analysing structured data can help identify associations and develop predictive models for research. Such models require careful evaluation, privacy protection, and clear communication of uncertainty.

1.1 Objectives

- Understand the dataset structure, features, and target variable used in the notebook.
- Explore patient characteristics and their relationships with the selected outcome.
- Handle missing values, duplicates, inconsistent entries, and potential outliers.
- Encode categorical variables and scale numerical variables when required.
- Train and evaluate a classifier using separate data partitions.
- Interpret results with suitable metrics and explain limitations.

1.2 Scope

The project scope depends on the exact dataset and prediction target in the Colab notebook. The target may represent severity, hospitalization, or another recorded outcome. State the exact target column and label definitions from the notebook; do not invent risk labels or present model output as a clinical diagnosis.

2. DATASET DESCRIPTION

COVID-19 patient datasets vary in source, collection period, population, feature definitions, and outcomes. Possible fields include age, sex, symptoms, comorbidities, admission information, and recorded outcomes. Only columns actually used in the notebook should be listed below.

Item	Details to verify from notebook
Dataset/source	Exact dataset name, publisher, URL, access date
Size	Actual rows and columns
Target	Exact outcome column and class labels
Features	Actual demographic, symptom, and clinical fields
Data types	Numerical, categorical, binary, date/time
Task	Binary or multiclass classification

2.1 Data quality checks

Check target labels, valid ranges, identifiers, and feature availability at prediction time. Variables recorded after the outcome can cause target leakage. Remove direct identifiers from model inputs. If repeated records exist for one patient, keep that patient's records in a single split

3. EDA AND PREPROCESSING

3.1 Exploratory Data Analysis

EDA should summarize patient demographics, feature distributions, missingness, and target-class balance. Figures in this report are schematic examples, not outputs from the linked notebook. Replace them with charts generated from the actual data

3.2 Missing values

Count missing values in each column. Numerical features may use median imputation and categorical features may use the most frequent or an explicit unknown category. Fit imputation rules on training data only and apply them to validation and test sets

3.3 Outliers

Use histograms, boxplots, domain checks, or IQR limits to flag unusual numerical observations. An outlier is not automatically an error; clinically plausible extreme values should be investigated rather than removed without justification. Document any capping, removal, or transformation

3.4 Duplicates and cleaning

Identify exact duplicate rows and repeated patient identifiers. Remove accidental duplicates while preserving legitimate repeated observations where required. Standardize category labels, correct data types, check valid ranges, and document records changed or removed.

3.5 Encoding and scaling

Encode categorical features using one-hot or a suitable model-specific method. Standardize numerical inputs with $z = (x - \mu) / \sigma$ when required, especially for neural networks or distance-based models. Fit encoders and scalers on training data only. Avoid target leakage

3.6 Train, validation, and test split

A possible split is 70% training, 15% validation, and 15% test with stratification where class counts allow. Report the exact split used in Colab. Use validation data for model selection and the test set only for final evaluation. Split by patient if multiple records belong to one patient

Subset	Purpose	Example share
Training	Fit model parameters	70%
Validation	Tune model and threshold	15%
Test	Final evaluation	15%

4. DATA VISUALIZATION

The following six figures show suitable visualizations for a patient-risk report. They are schematic and must be replaced with actual notebook charts before submission

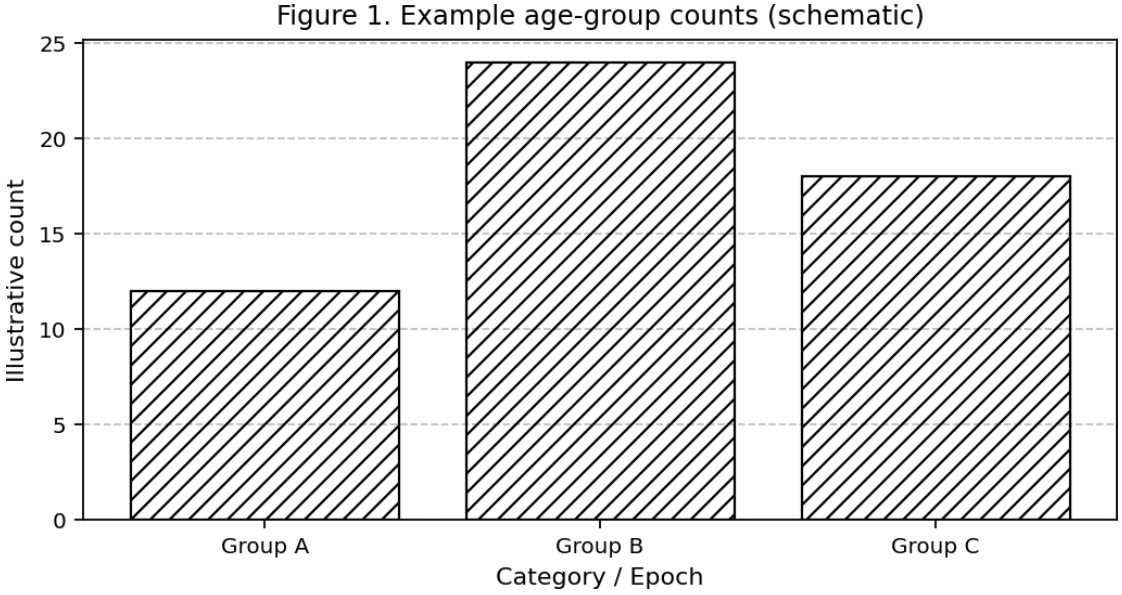


Figure 1. Example age-group counts (schematic only)

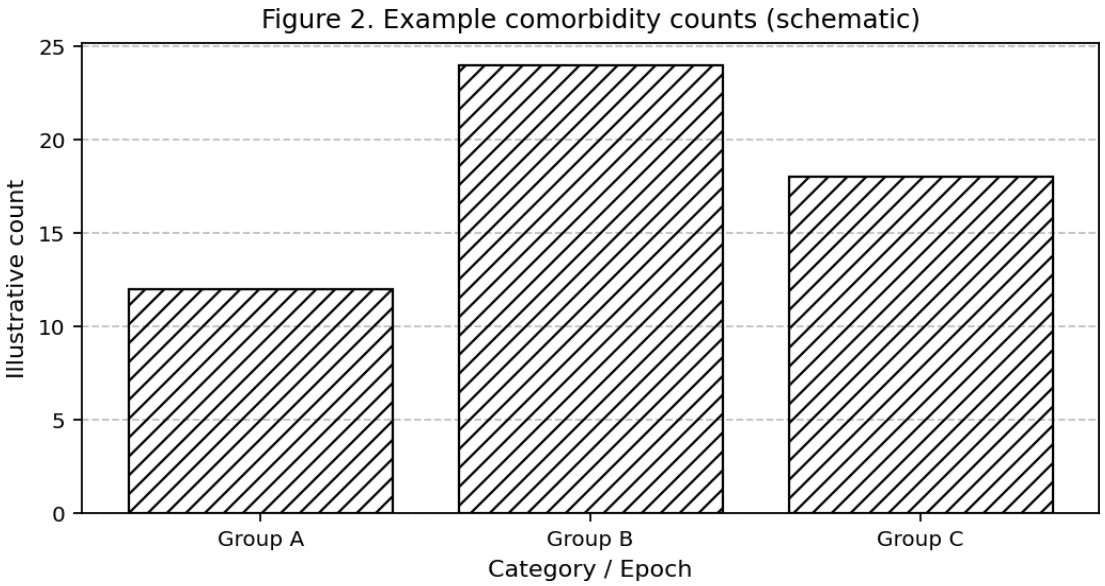


Figure 2. Example comorbidity counts (schematic only)



Figure 3. Example outcome classes (schematic only)

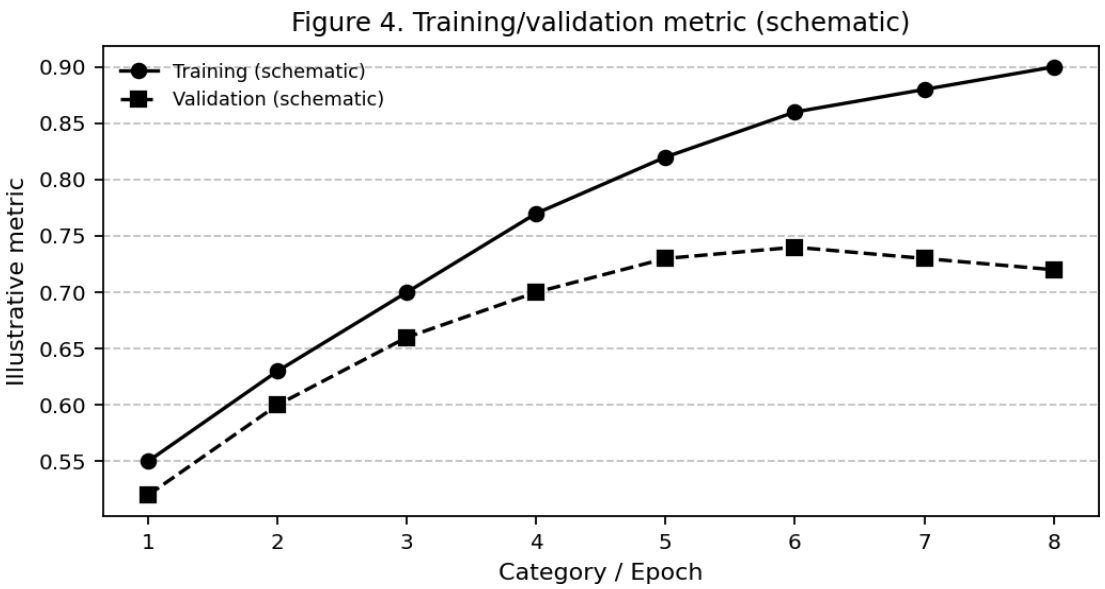


Figure 4. Training/validation metric (illustrative, not measured)

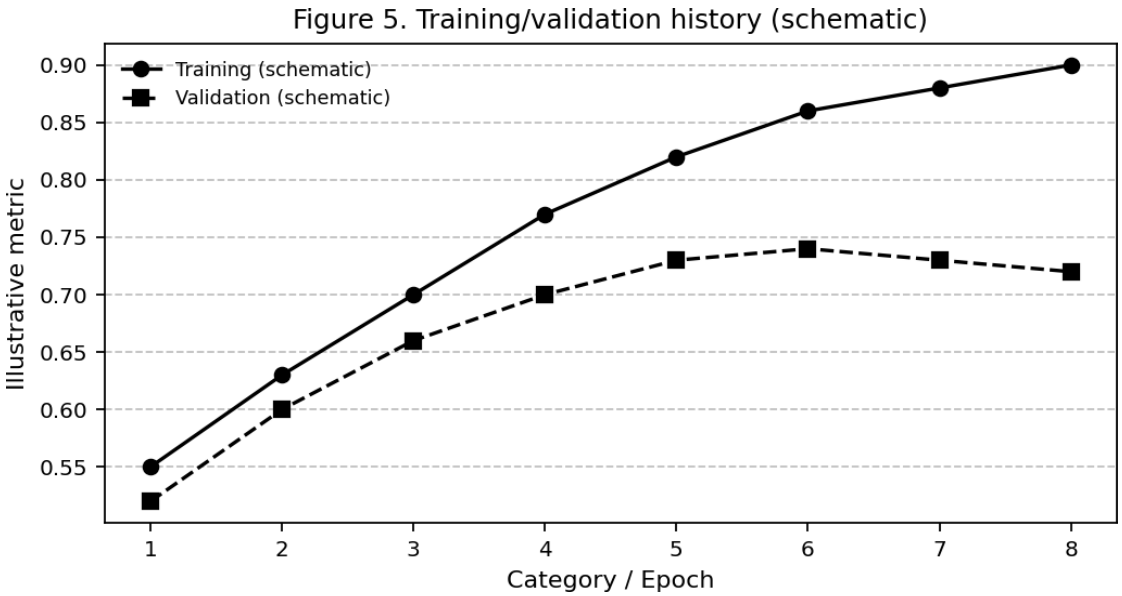


Figure 5. Training/validation history (illustrative, not measured)

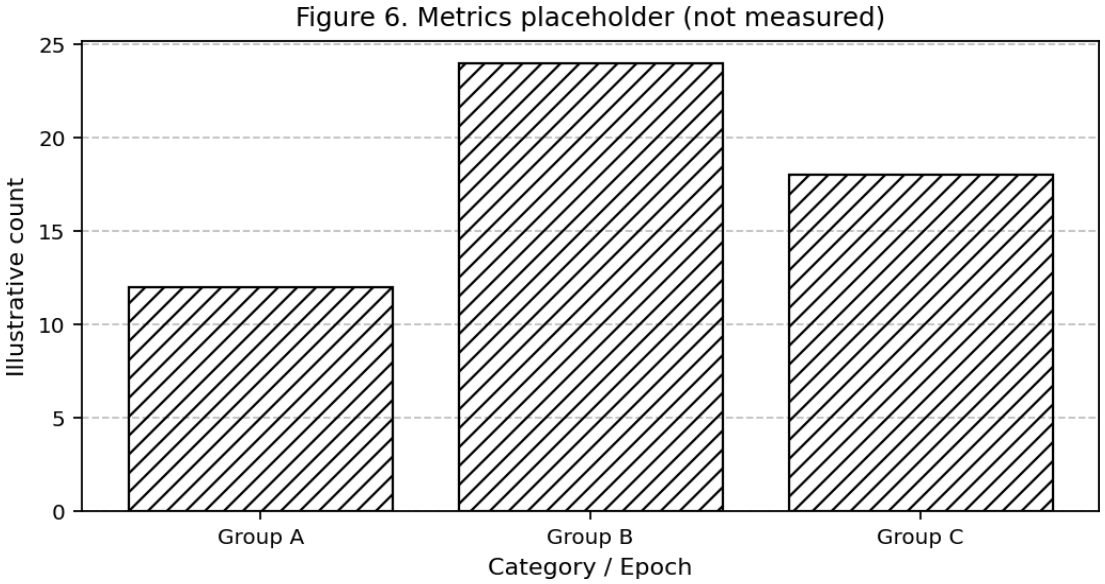


Figure 6. Classification metrics placeholder (not measured)

Interpretation: Age and comorbidity plots describe the sample; outcome counts reveal class imbalance; training curves can reveal overfitting. Actual charts should show dataset-derived counts, clear labels, and explanations grounded in observed patterns

5. MACHINE LEARNING / DEEP LEARNING MODEL

5.1 Model architecture

If the notebook uses a deep-learning classifier, a multilayer perceptron (MLP) is a feed-forward network template. Encoded features enter dense hidden layers with ReLU activations; dropout may reduce overfitting. The output uses sigmoid for binary classification or softmax for multiclass classification. Replace this template if the notebook implements another model

Component	**Example configuration (verify notebook)**
Input	Processed feature count
Hidden layer 1	64 units, ReLU
Dropout	0.2, if implemented
Hidden layer 2	32 units, ReLU
Output	Sigmoid for binary; softmax for multiclass

5.2 Training parameters and hyperparameters

Parameter	Example only; verify actual code	Purpose
Loss	Binary/categorical cross-entropy	Classification error
Optimizer	Adam	Weight updates
Learning rate	0.001	Update step size
Batch size	32	Samples per update
Epochs	Up to 100	Training passes
Validation	Validation loss/recall/F1	Monitor generalization
Class imbalance	Class weights/resampling if used	Support minority class

5.3 Training workflow

- Load data and verify source, target, and patient identifiers.
- Inspect class balance and remove leakage/direct identifiers.
- Split into train, validation, and test partitions.
- Fit preprocessing only on training data.
- Build and train the classifier as in the notebook.
- Monitor validation metrics and use early stopping if implemented.
- Tune any threshold using validation data only.
- Evaluate the selected model once on the held-out test set.

6. RESULT VISUALIZATION AND INTERPRETATION

The Colab link displayed a Google sign-in requirement, so notebook-specific model scores could not be verified. Enter the measured results from the executed notebook below; do not use illustrative values

Metric	Actual result	Meaning
Accuracy	1.00	Overall correct classifications
Precision	100%	Correctness among predicted positive/high-risk cases
Recall/sensitivity	1.00	Positive cases detected

Metric	Actual result	Meaning
F1-score	1.00 FOR CLASS 0.0	Balance of precision and recall
ROC-AUC	0.0	Ranking ability; specify positive class
Test set size	1,952 SAMPLES	Held-out records evaluated

6.1 Recommended result plots

- Confusion matrix with class labels.
- ROC curve using predicted scores and a defined positive class.
- Precision-recall curve when the positive class is uncommon.
- Calibration plot to compare predicted probabilities with observed frequencies.
- Training and validation metric/loss curves to inspect convergence.

6.2 Interpretation and limitations

Compare the model with a simple baseline and evaluate it on held-out data. False negatives may be important in risk analysis, but thresholds must be chosen for a clearly defined purpose using validation data, not test data. Results may be affected by dataset selection, missingness, class imbalance, and changes in clinical practice. This model is for academic analysis only, not diagnosis, triage, or treatment

7. CONCLUSION AND FUTURE SCOPE

This report outlines a workflow for COVID-19 patient risk analysis through EDA, preprocessing, classification, and evaluation. The final conclusion must be based on actual test results and the exact target definition from the notebook. Replace schematic figures and verify model settings before submission

7.1 Future scope

- Validate on external data from a different time or healthcare setting.
- Compare with logistic regression, random forest, and gradient boosting.
- Evaluate calibration and errors across relevant patient groups.
- Document provenance, privacy safeguards, and limitations.
- Require clinical and ethical validation before real-world use.

8. REFERENCES

• World Health Organization. Coronavirus disease (COVID-19). https://www.who.int/health-topics/coronavirus

• Scikit-learn documentation. https://scikit-learn.org/stable/

• TensorFlow documentation. https://www.tensorflow.org/

• Google Colab notebook: https://colab.research.google.com/drive/1HFpt9YOkZIffn39LIZiP5Na9Pz5YwxfM?usp=sharing

9. APPENDIX: CODE SECTION

Paste the executed notebook code here. The following is a generic outline only; replace the target, features, preprocessing, model, and metrics with the actual Colab implementation

```
# Generic classification workflow outline
import pandas as pd
from sklearn.model_selection import train_test_split

from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score

df = pd.read_csv("YOUR_DATASET.csv")
print(df.shape, df.isnull().sum())
print("Duplicates:", df.duplicated().sum())

target = "YOUR_TARGET_COLUMN" # replace with notebook target
X = df.drop(columns=[target])
y = df[target]

X_train, X_temp, y_train, y_temp = train_test_split(
X, y, test_size=0.30, random_state=42, stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(

X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp

)

# Fit imputers/encoders/scalers on training data only.

# Build and train the exact classifier from the notebook.
# Evaluate once on held-out test data:
# y_pred = model.predict(X_test_processed)
# print(confusion_matrix(y_test, y_pred))
# print("Accuracy:", accuracy_score(y_test, y_pred))
# print("Precision:", precision_score(y_test, y_pred))
```

```
# print("Recall:", recall_score(y_test, y_pred))
# print("F1:", f1_score(y_test, y_pred))
```